# Subscriber Pipeline Writeup

This subscriber pipeline project is one of my main portfolio projects from Codecademy. This project is a Data Engineering pipeline that transforms a “mock database of long-term cancelled subscribers for a fictional subscription company” (Codecademy) . The initially given data was two CSVs. One CSV was called cademycode and the other CSV was called cademycode_updated. The first CSV contains 5000 rows of student table rows whilst the second file contains an additional 1000 rows of student data. The end goal was to have a BASH script that was mostly automated run, clean, validate, and provide an updated, clean, database that would be directed to a ‘production’ directory.

The first task that needed to be done was an initial inspection of the main database (cademycode). Upon inspection, all of the data seemed to be clean. The database had three tables: cademycode_courses, cademycode_student_jobs, cademycode_students. There were instances of null values in the cademycode_students table, but because the nans were MAR (missing at random) I decided to keep them. Directly from my jupyter notebook comments the nulls we have are “#5 unemployed, 251 yet to complete a course, 471 with no career path, 471 with no hours studying.” which displays deeper insight into why the data is null. Also, in the same jupyter notebook I combine all of the tables into one merged dataframe and send the dataframe to a CSV, so that an analytics team could analyze the data. Though that was just a side goal, the main goal is having a cleaned SQL database. Please see Initial-Transforming.py for all of the code pertaining to this section of work.

With having now a clean, baseline database, the next order of business is to create the pipeline. Now to see the official order of the data pipeline please see run_pipeline.sh. Though, the first code written was test-pipeline.py which is composed of three unit tests. By importing unittest libraries into python I managed to set up lines of code that tested four distinct events: do the original (cademycode) and updated

(cademycode_updated) databases have the same table schema? Can table joins initiate properly in the updated database? Does the new database have updated rows present that the original does not already have? And are there duplicate uuids (cleaned version = student_id) present in the updated database? If any of the tests fail an error message gets sent to an error log and the whole pipeline stops. Also, please note there are some additional data validation functions in test-pipeline.py. The additional functions print information to the console about: new student records found, whether duplicate student uuids are found (similar to the unit test), and whether duplicate names are found in the student table. At the bottom of the python script I have a test runner which runs the test-pipeline where: loadTestsFromTestCase(TestDatabaseUpdates) collects the unit tests, TextTestRunner runs all tests and displays them in the console, result.failures contains tests that ran but failed an assertion, result.errors contains tests that crashed unexpectedly, error_logger.error() writes the failures and their details to the error log, and sys.exit() reports failures to the operating system if any tests fail.

The second python script that was written is inspect-database.py. This part of the pipeline is the most straightforward of all the stages. If you look at run_pipeline.sh, inspect-database.py is actually the first step of the pipeline. This step as you can imagine is the initial filter which is supposed to quickly distinguish whether both databases are constructed of the same tables and columns. The code simply reads both table names and column names in SQL.

The last stage of the subscriber pipeline is where the original database gets updated with the new rows from the updated database. All of the code that happens in this stage is located in update-pipeline.py. In the update stage there are five main functions that happen: the database gets read in as a pandas dataframe, the dataframe gets cleaned, finds new students, validates the students, and saves the newly updated clean dataframe to a SQL database. The cleaning of the dataframe took place in order to change “uuid” to a more descriptive “student_id” column name; and also for certain column data types to be altered. The student validation ensures that the correct columns are present, no nulls in the rows, and that

no duplicated student ids are found. The save students function inserts the clean student table into the database you want updated. In the python file you will be able to see multiple comments that I made throughout the code to help clarify what inputs you need for functions as well. Lastly, once update-pipeline.py was completed I ran some quality checks to make sure the outputs were legit (commented out the code in the file so you can still see). The quality checks consisted of: checking for duplicate student ids, looking for missing student ids, checking database integrity, and to check all three table counts. Also, please note that all actions taken place during the update pipeline are recorded in the changelog.

In conclusion we have the whole pipeline process which starts with inspect-pipeline.py, to test-pipeline.py, and lastly update-pipeline.py. Changelog.log records update events and errors.log records errors that happen only in the 'test' python file. If all runs well the BASH script will copy the final updated database into a "production" folder. This project was by far the biggest project I have ever taken on. I learned a lot about many facets of data engineering which I'm sure will stick with me for a long time. I did have help from many sources along the way, most notably ChatGPT; but yet I used the culmination of my Codecademy skills to really push me through completion!